

CIS007-3 Comparative Integrated Systems – AY 25/26

Student Name: Minal Pariyar

Student Number: 2214714

Student Signature:

Date:

Introduction

This project's primary goal is to create and build a working heart game that exemplifies knowledge of integrated software systems and fundamental concepts such virtual identity, interoperability, low coupling and high cohesion, and event-driven development. The project demonstrates a full-stack application developed using a React TypeScript frontend and a Node.js, Express and Mongo db backend architecture. Additionally, the following artifact retrieves and processes the Heart Game question via a third-party API made for Marc Coned.

Software Architecture (Low Coupling / High Cohesion)

The modular client-server architecture used in the following heart game was created to maintain strong cohesion and minimal coupling. Instead of trying to implement everything in a single file, each feature is broken out into several components to do a specific task, which helps with code maintenance, reusability, and scalability.

Backend Architecture:

To preserve high cohesiveness, I divided the backend codebase into many folders according to its functions. Controllers, routes, databases, models, middleware, and utils are the folders I have divided. First, the database connection logic with MongoDB is included in the db directories. In addition, I have divided the user and the game into two distinct entities, with the user-related characteristics in the user file and the game-related features in the game file. This promotes strong cohesiveness since each file concentrates on a specific task and low coupling since these two files won't be dependent on one another.

Additionally, I have divided the user and game in each folder. The user.controller file in the controller directory manages the user's fundamental functions, whereas the game.controller file manages game-related functions. Additionally, I have established RESTful endpoints in the routes, such as /api/game to manage game data and score, and /api/user to concentrate on user-related activities. In a similar vein, I've introduced auth middleware in the middleware section, which handles authentication tasks like confirming the JWT token.

Similarly, app.js and index.js files are responsible for initializing the application, connecting routes and middleware, and starting the server. Hence, this is the model architecture utilized in the backend, which results in low coupling because each component, such as the controller, models, and routes, operates independently, and high cohesion because each file focuses on independent responsibility.

Frontend Architecture:

The frontend is built using React with TypeScript, which is a JavaScript library for building user interfaces. In the frontend also I have separated several components and maintained modular architecture, which supports scalability and modularity. Here, App.tsx serves as the component that integrates routes, context providers, and global layout. Similarly, I have separated the game and user folders, where each directory contains components, which handle reusable UI components; hooks, which are custom React hooks for handling API data; services, which handle the URL endpoints; and types, which contain TypeScript interfaces and data models to ensure type safety.

Overall, both the frontend and backend follow the frontend modular architecture, which ensures high cohesion and low coupling.

Event Driven Architectures

The provided system heavily depends on event driven programming which means both frontend and backend respond to real time events triggered by users and system. In the frontend, events are generated through user interactions and application states.

Button clicks: Trigger function such as login, logout, register and submitting game

Tanstack React query and axios: Both react query and axios are responsible for invoking backend api calls. For instance react query helps in automatic updates and state management.

Routing Events: Navigation between pages triggers state updates and api calls.

In similar vein, in the backend events are represented by HTTP request. When a user performs an action, such as submitting a score:

- The frontend sends an HTTP request to a defined route (e.g., /api/game/submit).
- The route triggers an event handled by game.controller.js.
- Middleware functions act as interceptors, verifying authentication and input validity.
- Controllers process the logic, interact with models, and return a response event back to the client.

This reactive flow of request–response cycles represents an event-driven pattern where every user or system action triggers a specific response chain

Interoperability

Interoperability allows the system to communicate and exchange data and resources with other systems. Here, I've included a third-party PHP API service developed by Marc Conrad into my heart game. I've included that API into my backend, which uses the HTTPS protocol and exchanges data in JSON format. I'm receiving the question, answer, and carrots from the API. I used Axios to retrieve the heart game API on the backend, and I saved the result in my game document. Additionally, I have given this

function the name "create game" and made a different route for it. The frontend will utilize this to access the question.

So the basic flow is that the frontend calls the API name "create game." Following that, the backend fetches the heart game API to get the new game, and that game is stored in the game document and passed to the frontend. Hence, the question is received by the frontend. So this flow demonstrated the strong interoperability as it seamlessly connected with the frontend, backend, and third-party API services.

Virtual Identity

The system implements virtual identity management through a secure, token-based authentication mechanism using Access Tokens and Refresh Tokens, which are stored in HTTP-only cookies. This approach ensures that user sessions are both persistent and protected against common web vulnerabilities.

Using access and refresh tokens that are securely saved in cookies, the system incorporates secure virtual identity. In this case, I've made use of an access token, which allows users to access restricted API routes. The auth middleware I developed is in charge of confirming the user's identity for each request. In a similar vein, access tokens may be renewed using a refresh token, which eliminates the need for frequent log-ins. As a result, HTTP-only cookies are used to hold both tokens.

Thus, the backend grants access and updates the token once a user signs in. The generated tokens are immediately saved as secure cookies. In a similar manner, every protected API call has its token verified by auth middleware. The refresh token is used to create a new token in the event that the access token expires. Thus, secure user sessions are aided by this kind of authentication.

Any other interesting features

- Real-Time Score Tracking:

The system updates total, average, and accuracy scores instantly after each game, giving players immediate performance feedback.

- Recent Games History:

Displays previous challenges with user answers, correct results, and color-coded outcomes to help users review progress.

- Dynamic Visual Challenges:

Each question is uniquely generated from the external API, displaying engaging emoji-based puzzles for visual reasoning.

- Player Profile & Active Status:

Shows user information, email, and active session status, along with secure logout to maintain privacy and control.

- Reset Score Options:

Users can reset their progress or start a new challenge instantly without reloading the page for a seamless experience.